

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 12: Practical Exercises

Foreign Trade University

Instructions

For each problem below there is a starter file `exercises/starter_code_problemNN.py` containing function signatures with `raise NotImplementedError` bodies, and a small `__main__` block with tests. Implement the missing functions so the tests pass. A reference solution is provided in `solutions/solution_problemNN.py` – try the problem yourself before looking!

All problems use only the Python 3.10+ standard library. Shared helper functions live in `starter_code.py` at the week root: `num_variables`, `all_assignments`, `generate_3sat`, `make_graph`, `generate_graph`, `neighbors`, `complement_graph`, `is_clique`, `generate_subset_sum`, and `generate_distance_mat`.

A *CNF formula* is a list of clauses, each clause a list of signed integers (a positive literal v means “variable v is true,” $-v$ means false); a *graph* is a pair (n, edges) with `edges` a set of `frozenset({u, v})`. These problems concern *intractable* (NP-hard) problems, so the reference algorithms are deliberately exponential brute-force / backtracking searches – correct, but only practical for small inputs.

The 12 problems are grouped into three themes:

- **Verifiers & Brute Force** (Problems 1–5): SAT verification and brute-force solving; verifiers and brute-force optima for independent set, vertex cover, and clique, including the complementarity identities $VC = V \setminus IS$ and “max clique in $G = \max$ independent set in $\overline{G}$.”
- **Reductions** (Problems 6, 12): the gadget reduction $3\text{-SAT} \rightarrow \text{INDEPENDENT SET}$, and an explicit mapping reduction between `VERTEX COVER` and `INDEPENDENT SET` with the equivalence “ $x \in A \iff f(x) \in B$ ” checked on random instances.
- **Search** (Problems 7–11): brute-force `SUBSET SUM`; backtracking `HAMILTONIAN CYCLE`; exact TSP via Held–Karp bitmask DP (verified against brute force); backtracking graph k -coloring and the chromatic number.

1 Verifiers & Brute Force

Problem 1 – SAT Verifier

Implement `verify_sat(formula, assignment)`: given a CNF formula (list of clauses, each a list of signed ints) and an `assignment` (a tuple/list of booleans where `assignment[v-1]` is the value of variable v), return `True` iff the assignment satisfies the formula – i.e. every clause has at least one true literal. A literal $v > 0$ is true iff `assignment[v-1]` is `True`; a literal $v < 0$ is true iff `assignment[-v-1]` is `False`.

Example. `verify_sat([[1, -2], [2, 3]], (True, False, False)) == True`. The empty formula `[]` is satisfied by any assignment; a formula containing the empty clause `[]` is satisfied by none.

Problem 2 – Brute-Force SAT Solver

Implement `brute_force_sat(formula)`: try all 2^n assignments over the $n = \text{num_variables}(\text{formula})$ variables (use `all_assignments` from `starter_code.py`) and return the first satisfying assignment found (as a tuple of booleans), or `None` if the formula is unsatisfiable. This is the exponential “certificate search” that the verifier of Problem 1 short-circuits when a certificate is already known.

Problem 3 – Independent-Set Verifier and Maximum Independent Set

Implement `is_independent_set(graph, subset)`: return `True` iff no two vertices of `subset` (an iterable of vertex labels) are joined by an edge of `graph`. Then implement `max_independent_set(graph)`: by checking all 2^n vertex subsets (largest first), return a maximum independent set as a `frozenset`. Intended for $n \leq 18$.

Problem 4 – Vertex-Cover Verifier, Minimum Vertex Cover, and Complementarity

Implement `is_vertex_cover(graph, subset)`: return `True` iff every edge of `graph` has at least one endpoint in `subset`. Implement `min_vertex_cover(graph)`: by brute force, return a minimum vertex cover as a `frozenset`. Finally implement `verify_vc_complement(graph)`: return `True` iff $V \setminus (\text{a maximum independent set})$ is a minimum vertex cover – i.e. the sizes satisfy $|VC| = n - |IS|$ and the complement of a maximum independent set is in fact a valid vertex cover.

Problem 5 – Clique Verifier, Maximum Clique, and the Complement Identity

Implement `is_clique_subset(graph, subset)`: return `True` iff every pair of vertices in `subset` is joined by an edge. Implement `max_clique(graph)`: by brute force, return a maximum clique as a `frozenset`. Finally implement `verify_clique_is_complement(graph)`: return `True` iff the size of a maximum clique in `graph` equals the size of a maximum independent set in `complement_graph(graph)` (using `max_independent_set` from Problem 3).

2 Reductions

Problem 6 – Reduction 3-SAT $\rightarrow$ Independent Set

Implement `reduce_3sat_to_independent_set(formula)` returning `(graph, k)`:

- Create one vertex per (clause, literal-position) pair, so a formula with k clauses of ≤ 3 literals each yields up to $3k$ vertices.
- Add *triangle edges*: within each clause, join all of its literal-vertices pairwise.
- Add *conflict edges*: between any two vertices in *different* clauses whose literals are complementary (one is v , the other $\neg v$).
- Set k = number of clauses.

Then implement `verify_reduction(formula)`: return `True` iff `brute_force_sat(formula)` finds the formula satisfiable *exactly when* the constructed graph has an independent set of size $\geq k$ (compute the latter with `max_independent_set` from Problem 3). This is the both-directions correctness check of the reduction, done computationally on small formulas.

Tip. You may import `brute_force_sat` (Problem 2) and `max_independent_set` / `is_independent_set` (Problem 3) – the reference solution re-implements the small pieces it needs so each file is self-contained.

Problem 12 – Reduction Certificate Demo: Vertex Cover $\leftrightarrow$ Independent Set

Implement a poly-time mapping reduction and verify the certificate equivalence “ $x \in A \iff f(x) \in B$ ” on random small instances.

Define the decision predicates `has_independent_set(graph, k)` (does `graph` have an independent set of size $\geq k$?) and `has_vertex_cover(graph, k)` (size $\leq k$?), each by brute force. Implement the mapping `reduce_is_to_vc(graph, k)` returning `(graph, n - k)` (leave the graph unchanged, complement the threshold), reflecting the identity “ G has an IS of size $\geq k$ iff G has a VC of size $\leq n - k$.” Finally implement `verify_mapping(num_trials, n, seed)`: for `num_trials` random graphs and a sweep of thresholds k , confirm `has_independent_set(graph, k)` equals `has_vertex_cover(*reduce_is_to_vc(graph, k))` for every $(graph, k)$. Return `True` iff the equivalence holds in every case.

3 Search

Problem 7 – Subset-Sum Verifier and Brute-Force Decision

Implement `verify_subset_sum(weights, subset_indices, target)`: return `True` iff the items of `weights` at the given indices sum to exactly `target`. Then implement `subset_sum_exists(weights, target)`: by checking all 2^n subsets, return `True` iff some subset of `weights` sums exactly to `target`. Also return a witness via `find_subset_sum(weights, target)`, which returns a list of indices summing to `target`, or `None` if none exists.

Example. `subset_sum_exists([3, 34, 4, 12, 5, 2], 9) == True` ($4 + 5 = 9$).

Problem 8 – Hamiltonian Cycle by Backtracking

Implement `verify_hamiltonian_cycle(graph, cycle)`: given a candidate `cycle` (a list of vertices, *without* repeating the start at the end), return `True` iff it visits every vertex exactly once and consecutive vertices (including the wrap-around from last to first) are joined by edges. Then implement `find_hamiltonian_cycle(graph)`: search by backtracking for a Hamiltonian cycle, returning it as a list of vertices (starting at vertex 0), or `None` if the graph has none.

Example. The 4-cycle $0-1-2-3-0$ has a Hamiltonian cycle; a “path” graph $0-1-2-3$ (no edge $3-0$) does not.

Problem 9 – TSP via Held–Karp Bitmask DP

Implement `tsp_held_karp(dist)`: given an $n \times n$ symmetric distance matrix, return the length of a shortest tour visiting every city exactly once and returning to the start (city 0), using the Held–Karp dynamic program over subsets:

```
dp[S][j] = shortest path that starts at 0, visits exactly the cities in S,  
           and ends at j (where 0 in S, j in S).  
answer = min over j != 0 of dp[FULL][j] + dist[j][0].
```

This runs in $O(n^2 2^n)$ time. Then implement `tsp_brute_force(dist)` that tries all $(n-1)!$ permutations of the non-start cities, and `verify_tsp(num_trials, n, seed)` returning `True` iff Held–Karp and brute force agree (within 10^{-9}) on `num_trials` random matrices.

Problem 10 – Graph k -Coloring by Backtracking

Implement `verify_coloring(graph, coloring)`: given `coloring` (a list/dict mapping each vertex to a color index), return `True` iff it uses at most the intended colors and no edge joins two equally-colored vertices. Then implement `k_coloring(graph, k)`: search by backtracking for a proper coloring of `graph` with colors $\{0, \dots, k-1\}$, returning it as a list (indexed by vertex) or `None` if no proper k -coloring exists.

Example. A triangle K_3 needs 3 colors: `k_coloring` returns `None` for $k = 2$ and a valid coloring for $k = 3$.

Problem 11 – Chromatic Number (Trying $k = 1, 2, 3, \dots$)

Implement `chromatic_number(graph)`: using `k_coloring` from Problem 10, return the smallest $k \geq 0$ for which `graph` has a proper k -coloring (a graph with no edges and ≥ 1 vertex needs 1 color; the empty graph on 0 vertices needs 0). Then implement `is_three_colorable(graph)` returning

True iff $\text{chromatic_number}(\text{graph}) \leq 3$. *Sanity check (reduction intuition)*: a bipartite graph (e.g. an even cycle) has chromatic number ≤ 2 , while an odd cycle has chromatic number 3.

4 LeetCode Practice

After completing the problems above, practise this week's intractability techniques (verification, reductions, and exponential backtracking / bitmask DP for NP-hard problems) on the following [LeetCode](#) problems. NP-complete problems are attacked here with exponential backtracking / branch-and-bound. Difficulty is LeetCode's own label.

- [Subsets](#) – *Medium* – Backtracking.
- [Combination Sum](#) – *Medium* – Backtracking.
- [Palindrome Partitioning](#) – *Medium* – Backtracking.
- [Word Search](#) – *Medium* – Backtracking.
- [N-Queens](#) – *Hard* – Backtracking.
- [Sudoku Solver](#) – *Hard* – Constraint backtracking.
- [Partition to K Equal Sum Subsets](#) – *Medium* – NP-hard (subset-sum).
- [Flower Planting With No Adjacent](#) – *Medium* – Graph coloring.
- [Find the Shortest Superstring](#) – *Hard* – TSP / bitmask DP.